Repair ledger — Histor-lite Aletheia Importer

Issues found and fixed during the build, in Project Witness's "make claims, attack
claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): Histor-lite Aletheia Importer

First Histor-lite Aletheia importer build — Move 16 of Project Witness. It takes a
verified Aletheia Evidence Bundle v0.1 and records it as a Histor-lite-style
evidence case (summary + manifest copy + replay report + checksums), preserving
Aletheia's integrity status exactly, and verifies that case later. It is not
connected to ODIN, Coordinator, or Companion.

Scope held deliberately narrow: one Aletheia bundle -> one evidence case record;
metadata import only; no evidence-file copy, no ODIN import, no natural language, no

LLM, no truth claims, no signing, no anchoring. The importer records integrity metadata only — it never claims the evidence contents are true.

Design decisions worth recording:

- **Integrity status preserved exactly.** The summary's `source_evidence_status` is copied verbatim from the Aletheia manifest, and verify requires both the summary and the copied manifest to read `VERIFIED_PRESENT`. The source and bundled SHA-256 values are likewise carried through and cross-checked against the manifest copy.
- **Truth-claim guard at verify.** The replay report records `truth_claim: none` plus an explicit disclaimer sentence, and verify scans BOTH the summary and the replay report for forbidden truth/authenticity tokens (authentic, proven_true, source.trusted, file.authentic, etc.), refusing with `TRUTH_CLAIM` if any appears — even after the checksums are resynced.
- **Verify recomputes cross-references; it does not trust the summary.** It confirms case_id and evidence_id match across the summary and manifest copy, the source and

bundled SHAs match the manifest copy, the source status is VERIFIED_PRESENT, and the import status is IMPORTED_VERIFIED_ALETHEIA — each after a canonical + hash check, so resyncing the checksums after a lie does not help.

- **Exactly four files, nothing else.** The case folder must contain only the four named files; verify refuses any unknown extra file, and a test asserts no `evidence/` dir or `.bin` is written (the evidence file is never copied).
- **Strict, lowercase-only checksums.** Exactly three keys; duplicate/missing/unknown keys fail; malformed digests fail; uppercase digests fail.
- **Frozen boundary is injectable.** Aletheia verify is called via subprocess at runtime, but the unit suite injects an in-process fake verifier, so the freeze gate spawns no subprocesses and does not require the real frozen Aletheia folder. The subprocess path is exercised only by the documented manual smoke test.
- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output

directory; `require()` not bare
assert; in-process selfcheck that never launches
the unit suite.

Minor self-fix during the build: one reader test initially expected
`ManifestError` for an unsafe evidence_id, but `validate_id` raises
`ValidationError` (a sibling subclass of `HistorImportError`); the test was
corrected to assert the shared base type, since the contract is simply that an
unsafe id is refused.

Real end-to-end integration confirmed. Against frozen Aletheia v0.1: built a
real bundle, `import-evidence` produced `IMPORTED_VERIFIED_ALETHEIA` with
`source_evidence_status = VERIFIED_PRESENT`, both SHAs preserved, exactly the four
case files written, `truth_claim: none` in the replay report; `verify-case` passed.

Release gate (lesson applied directly): no subprocess orchestrator in the freeze
gate. The authoritative gate is the explicit command list in RELEASE_COMMANDS.md
plus the static `verify_release_manifest.py` (launches no subprocesses; additionally
checks the README scopes the importer to integrity

metadata only, disclaims proving truth, says it does not import ODIN reasoning, and says it is not connected to Coordinator or Companion yet).

Verified: all release-gate commands pass individually from a clean extraction (see TEST_RESULTS.md); selfcheck 8/8 in-process gates; verify_release_manifest.py passes (10/10, no subprocesses); tamper to any case file, the checksums, the file set, the cross-referenced IDs/statuses/SHAs, or an injected truth claim is detected with the correct reason.